Reinforcement Learning for LLMs

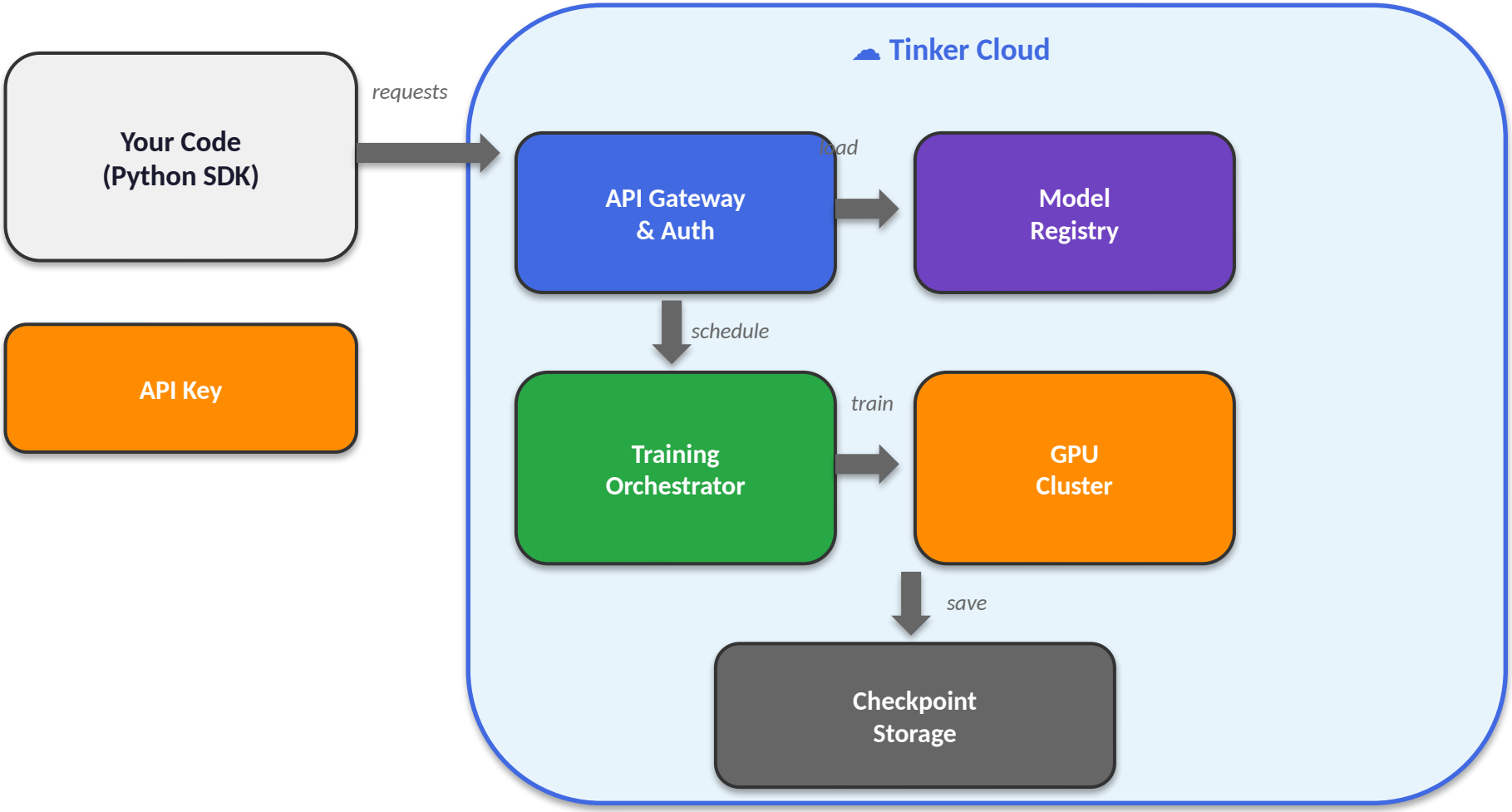
Training Language Models with Tinker

PES LLM Research Project

What is Tinker?

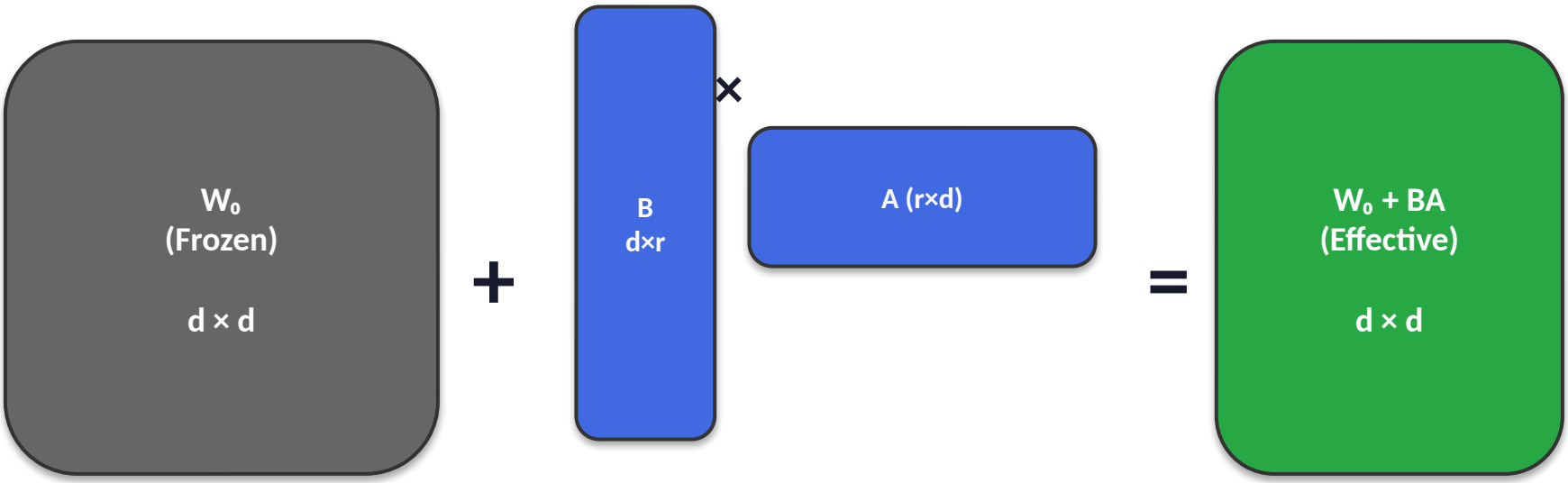
- Cloud-based RL training platform for LLMs
- Supports LoRA fine-tuning with reinforcement learning
- Pre-built recipes: Math RL, Preference, Distillation
- No GPU required locally - training runs in the cloud
- Python SDK with simple API

How Tinker Works: Cloud Architecture



Tinker handles GPU allocation, model loading, and checkpointing - you just write training code

LoRA: Low-Rank Adaptation



Original model weights (billions)

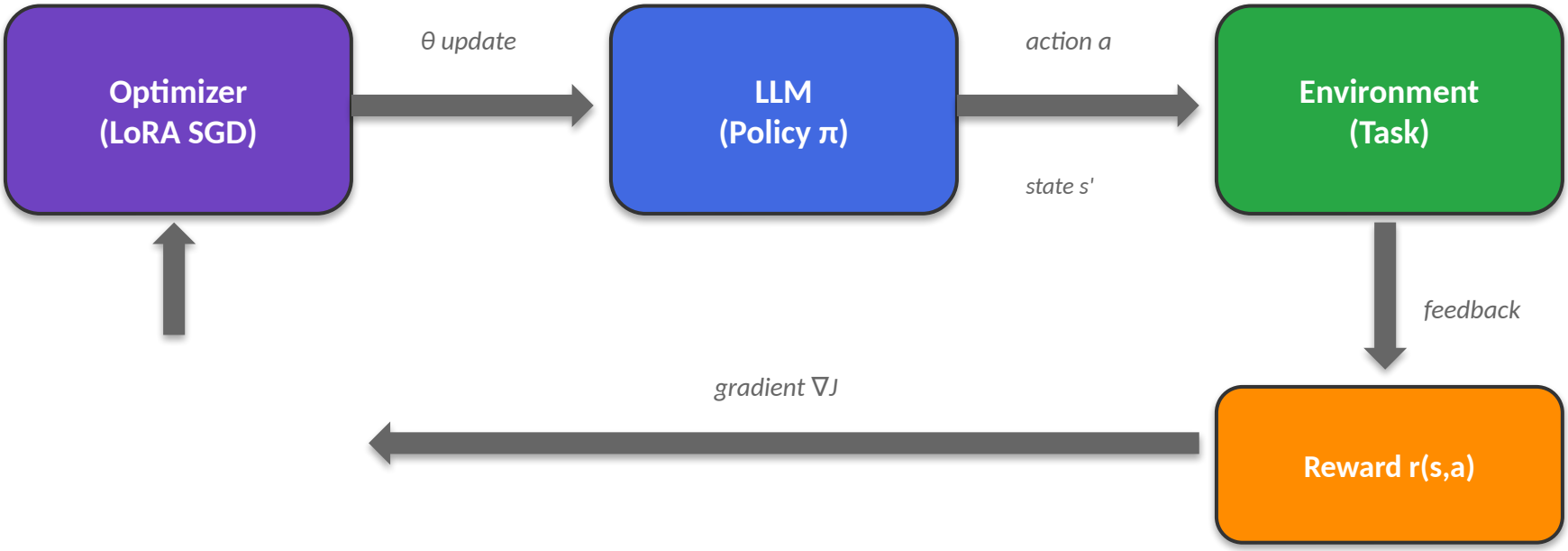
LoRA adapters (millions)

Combined for inference

rank r << d: Train only ~0.1% of parameters while preserving model capabilities

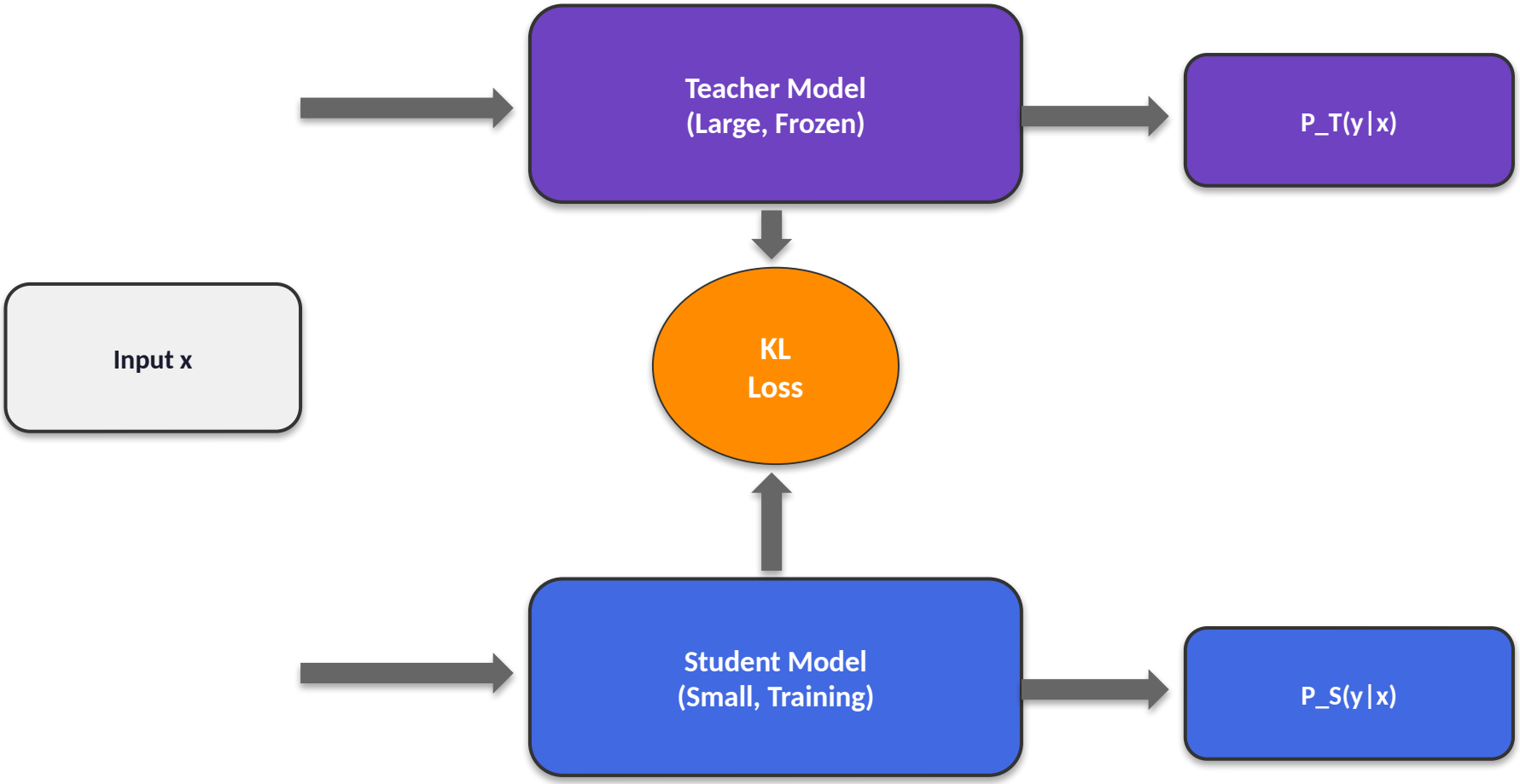
Example: d=4096, r=32 → 99.2% fewer trainable parameters

Reinforcement Learning Training Loop



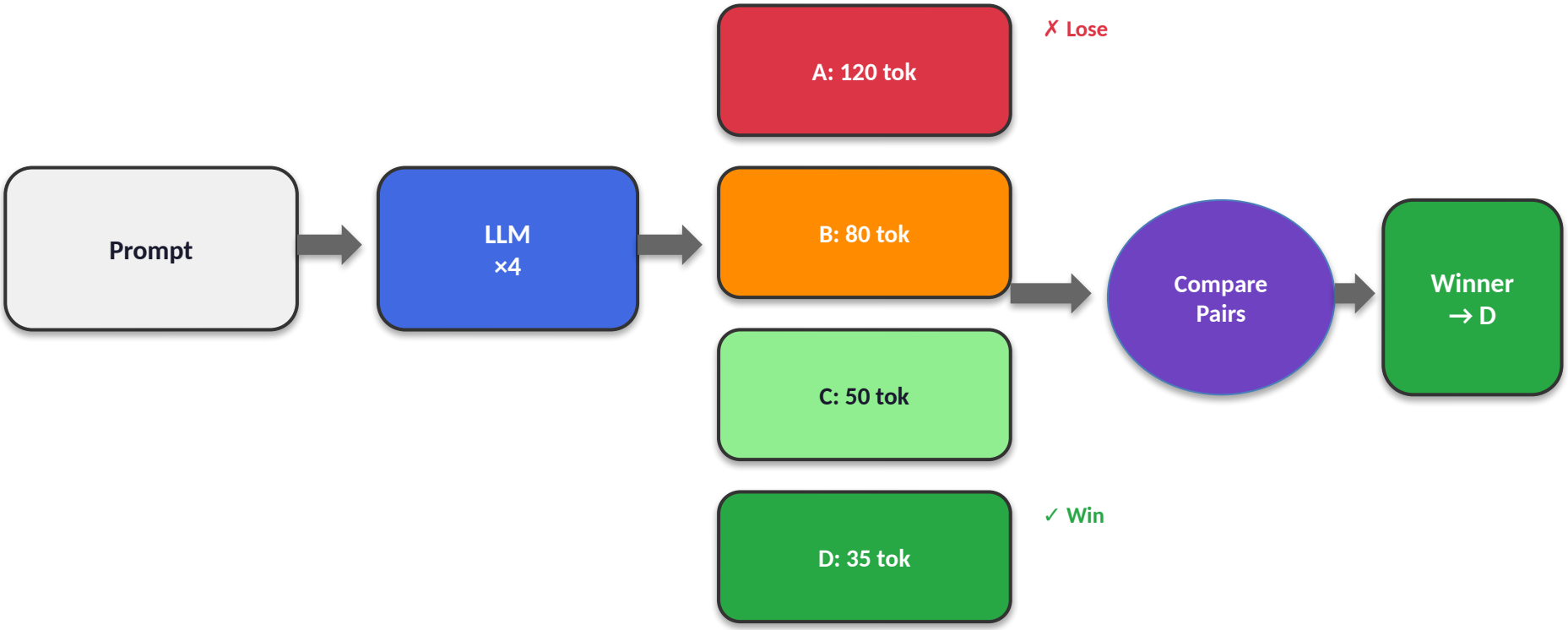
Objective: $J(\theta) = E[\Sigma\ r(s,a)] \rightarrow$ maximize expected reward

Knowledge Distillation: Teacher → Student



$$\text{Loss} = KL(P_T \,||\, P_S) = \Sigma\, P_T(y)\, \log(P_T(y)/P_S(y))$$

Pairwise Preference Learning



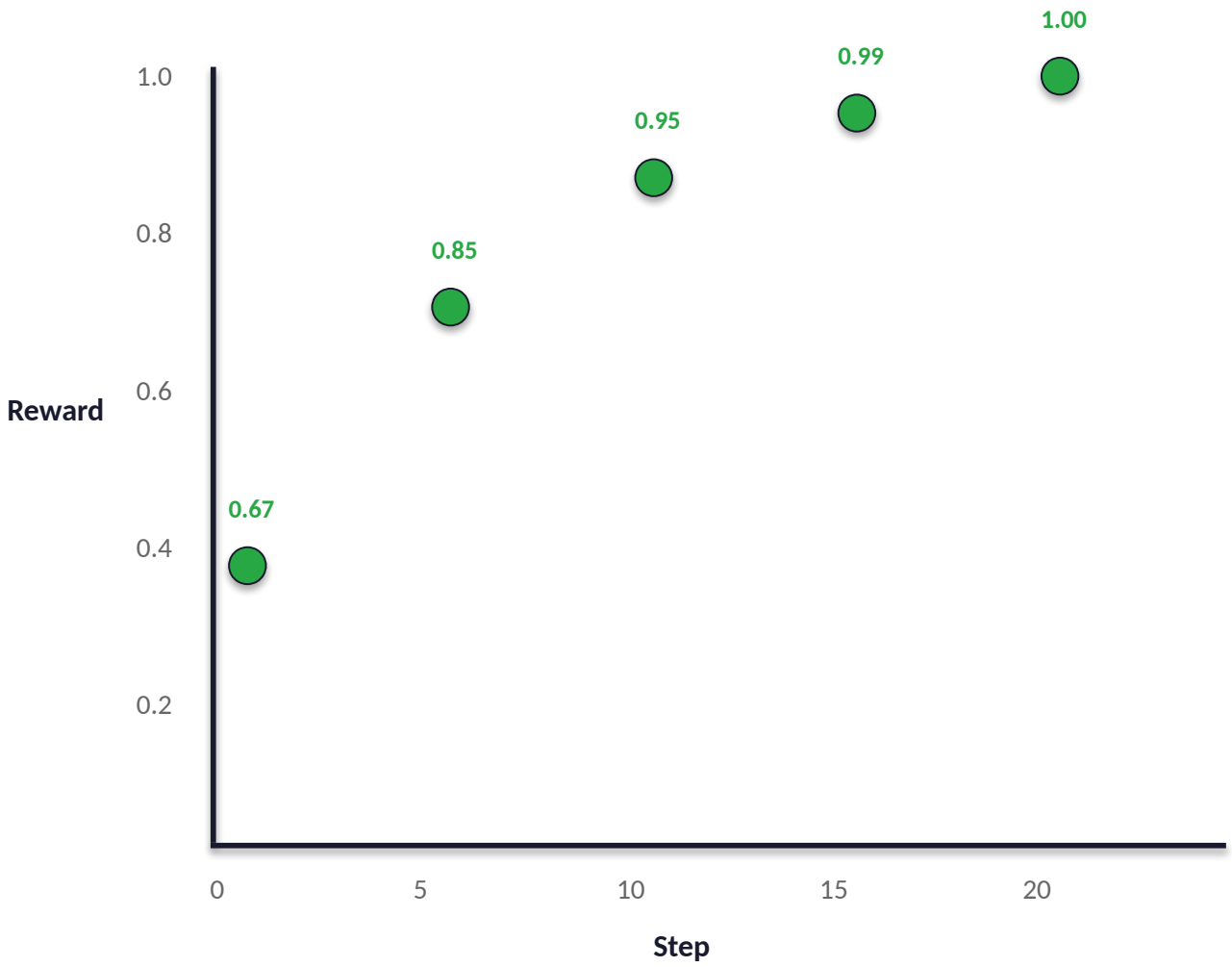

```
reward = (wins - losses) / comparisons + format_bonus
```

Shorter responses beat longer ones → model learns conciseness

Experiments Conducted

Recipe	Task	Model	Status	Result
Math RL	Arithmetic	Llama-3.2-1B	✓ Complete	100% accuracy
Chat SL	Conversation	Llama-3.2-1B	✓ Running	NLL: 2.5→1.8
Preference	Shorter	Qwen-0.6B	✓ Running	37% done
Distill (Off)	Reasoning	Llama-3.2-1B	✓ Running	SFT active
Distill (On)	KL Match	Llama-3.2-1B	✓ Running	KL: 2.88
GSM8K	Word Problems	Llama-3.2-1B	✓ Running	Multi-step

Math RL Results: Learning Curve



Key Results:

- 69.5% → 100%
- Converged in 20 steps
- ~2 min training
- Perfect accuracy

Example: Math RL Training

```
# Install Tinker
pip install tinker tinker-cookbook

# Set API key
export TINKER_API_KEY="your-key-here"

# Run RL training
python -m tinker_cookbook.recipes.math_rl.train \
    model_name="meta-llama/Llama-3.2-1B" \
    env=arithmetic \
    group_size=4 \
    groups_per_batch=100 \
    learning_rate=1e-4

# Output:
# Step 0:  reward=0.676, correct=69.5%
# Step 10: reward=0.998, correct=99.8%
# Step 20: reward=1.000, correct=100%
```

Jupyter Notebooks Created

- 01_math_rl_arithmetic.ipynb - Basic RL for addition
- 02_chat_sl_sft.ipynb - Supervised fine-tuning
- 03_preference_shorter.ipynb - Pairwise preference learning
- 04_distillation_off_policy.ipynb - SFT distillation
- 05_distillation_on_policy.ipynb - KL distillation
- 06_math_rl_gsm8k.ipynb - Word problem solving

Key Findings

- Simple tasks (arithmetic): RL achieves 100% in ~20 steps
- Complex tasks (GSM8K): Requires more training, larger models
- Preference learning: Effective for stylistic control
- Distillation: Efficient knowledge transfer
- Cloud training: Accessible without local GPUs

Future Directions

- Code generation with sandbox (DeepCoder)
- Multi-agent RL training
- RLHF with human preference data
- Scaling to larger models (8B, 32B)
- Custom reward functions

Thank You!

Questions?

pes-llm-research/tinker-experiments